

7.2.1 节练习

练习 7.20: 友元在什么时候有用? 请分别列举出使用友元的利弊。

练习 7.21: 修改你的 `Sales_data` 类使其隐藏实现的细节。你之前编写的关于 `Sales_data` 操作的程序应该继续使用, 借助类的新定义重新编译该程序, 确保其工作正常。

练习 7.22: 修改你的 `Person` 类使其隐藏实现的细节。

7.3 类的其他特性

虽然 `Sales_data` 类非常简单, 但是通过它我们已经了解 C++ 语言中关于类的许多语法要点。在本节中, 我们将继续介绍 `Sales_data` 没有体现出来的一些类的特性。这些特性包括: 类型成员、类的成员的类内初始值、可变数据成员、内联成员函数、从成员函数返回 `*this`、关于如何定义并使用类类型及友元类的更多知识。

7.3.1 类成员再探

为了展示这些新的特性, 我们需要定义一对相互关联的类, 它们分别是 `Screen` 和 `Window_mgr`。

定义一个类型成员

`Screen` 表示显示器中的一个窗口。每个 `Screen` 包含一个用于保存 `Screen` 内容的 `string` 成员和三个 `string::size_type` 类型的成员, 它们分别表示光标的位置以及屏幕的高和宽。

除了定义数据和函数成员之外, 类还可以自定义某种类型在类中的别名。由类定义的类型名字和其他成员一样存在访问限制, 可以是 `public` 或者 `private` 中的一种:

```
class Screen {
public:
    typedef std::string::size_type pos;
private:
    pos cursor = 0;
    pos height = 0, width = 0;
    std::string contents;
};
```

我们在 `Screen` 的 `public` 部分定义了 `pos`, 这样用户就可以使用这个名字。 `Screen` 的用户不应该知道 `Screen` 使用了一个 `string` 对象来存放它的数据, 因此通过把 `pos` 定义成 `public` 成员可以隐藏 `Screen` 实现的细节。

◀ 272

关于 `pos` 的声明有两点需要注意。首先, 我们使用了 `typedef` (参见 2.5.1 节, 第 60 页), 也可以等价地使用类型别名 (参见 2.5.1 节, 第 60 页):

```
class Screen {
public:
    // 使用类型别名等价地声明一个类型名字
    using pos = std::string::size_type;
    // 其他成员与之前的版本一致
};
```

其次, 用来定义类型的成员必须先定义后使用, 这一点与普通成员有所区别, 具体原因将

在 7.4.1 节（第 254 页）解释。因此，类型成员通常出现在类开始的地方。

Screen 类的成员函数

要使我们的类更加实用，还需要添加一个构造函数令用户能够定义屏幕的尺寸和内容，以及其他两个成员，分别负责移动光标和读取给定位置的字符：

```
class Screen {
public:
    typedef std::string::size_type pos;
    Screen() = default; // 因为 Screen 有另一个构造函数，
                        // 所以本函数是必需的
    // cursor 被其类内初始值初始化为 0
    Screen(pos ht, pos wd, char c): height(ht), width(wd),
    contents(ht * wd, c) { }
    char get() const // 读取光标处的字符
    { return contents[cursor]; } // 隐式内联
    inline char get(pos ht, pos wd) const; // 显式内联
    Screen &move(pos r, pos c); // 能在之后被设为内联
private:
    pos cursor = 0;
    pos height = 0, width = 0;
    std::string contents;
};
```

因为我们已经提供了一个构造函数，所以编译器将不会自动生成默认的构造函数。如果我们的类需要默认构造函数，必须显式地把它声明出来。在此例中，我们使用 `=default` 告诉编译器为我们合成默认的构造函数（参见 7.1.4 节，第 237 页）。

需要指出的是，第二个构造函数（接受三个参数）为 `cursor` 成员隐式地使用了类内初始值（参见 7.1.4 节，第 238 页）。如果类中不存在 `cursor` 的类内初始值，我们就需要像其他成员一样显式地初始化 `cursor` 了。

273 令成员作为内联函数

在类中，常有一些规模较小的函数适合于被声明成内联函数。如我们之前所见的，定义在类内部的成员函数是自动 `inline` 的（参见 6.5.2 节，第 213 页）。因此，`Screen` 的构造函数和返回光标所指字符的 `get` 函数默认是 `inline` 函数。

我们可以在类的内部把 `inline` 作为声明的一部分显式地声明成员函数，同样的，也能在类的外部用 `inline` 关键字修饰函数的定义：

```
inline // 可以在函数的定义处指定 inline
Screen &Screen::move(pos r, pos c)
{
    pos row = r * width; // 计算行的位置
    cursor = row + c; // 在行内将光标移动到指定的列
    return *this; // 以左值的形式返回对象
}
char Screen::get(pos r, pos c) const // 在类的内部声明成 inline
{
    pos row = r * width; // 计算行的位置
    return contents[row + c]; // 返回给定列的字符
}
```

虽然我们无须在声明和定义的地方同时说明 `inline`，但这么做其实是合法的。不过，最好只在类外部定义的地方说明 `inline`，这样可以使类更容易理解。



和我们在头文件中定义 `inline` 函数的原因一样（参见 6.5.2 节，第 214 页），`inline` 成员函数也应该与相应的类定义在同一个头文件中。

重载成员函数

和非成员函数一样，成员函数也可以被重载（参见 6.4 节，第 206 页），只要函数之间在参数的数量和/或类型上有所区别就行。成员函数的函数匹配过程（参见 6.4 节，第 208 页）同样与非成员函数非常类似。

举个例子，我们的 `Screen` 类定义了两个版本的 `get` 函数。一个版本返回光标当前位置的字符；另一个版本返回由行号和列号确定的位置的字符。编译器根据实参的数量来决定运行哪个版本的函数：

```
Screen myscreen;
char ch = myscreen.get();      // 调用 Screen::get()
ch = myscreen.get(0,0);       // 调用 Screen::get(pos, pos)
```

可变数据成员

有时（但并不频繁）会发生这样一种情况，我们希望能修改类的某个数据成员，即使是在一个 `const` 成员函数内。可以通过在变量的声明中加入 `mutable` 关键字做到这一点。

一个可变数据成员（mutable data member）永远不会是 `const`，即使它是 `const` 对象的成员。因此，一个 `const` 成员函数可以改变一个可变成员的值。举个例子，我们将给 `Screen` 添加一个名为 `access_ctr` 的可变成员，通过它我们可以追踪每个 `Screen` 的成员函数被调用了多少次：

◀ 274

```
class Screen {
public:
    void some_member() const;
private:
    mutable size_t access_ctr;    // 即使在一个 const 对象内也能被修改
    // 其他成员与之前的版本一致
};

void Screen::some_member() const
{
    ++access_ctr;                // 保存一个计数值，用于记录成员函数被调用的次数
    // 该成员需要完成的其他工作
}
```

尽管 `some_member` 是一个 `const` 成员函数，它仍然能够改变 `access_ctr` 的值。该成员是个可变成员，因此任何成员函数，包括 `const` 函数在内都能改变它的值。

类数据成员的初始值

在定义好 `Screen` 类之后，我们将继续定义一个窗口管理类并用它表示显示器上的一组 `Screen`。这个类将包含一个 `Screen` 类型的 `vector`，每个元素表示一个特定的 `Screen`。默认情况下，我们希望 `Window_mgr` 类开始时总是拥有一个默认初始化的

C++
11

Screen。在 C++11 新标准中，最好的方式就是把这个默认值声明成一个类内初始值（参见 2.6.1 节，第 64 页）：

```
class Window_mgr {
private:
    // 这个 Window_mgr 追踪的 Screen
    // 默认情况下，一个 Window_mgr 包含一个标准尺寸的空白 Screen
    std::vector<Screen> screens{Screen(24, 80, ' ')};
};
```

当我们初始化类类型的成员时，需要为构造函数传递一个符合成员类型的实参。在此例中，我们使用一个单独的元素值对 vector 成员执行了列表初始化（参见 3.3.1 节，第 87 页），这个 Screen 的值被传递给 vector<Screen> 的构造函数，从而创建了一个单元素的 vector 对象。具体地说，Screen 的构造函数接受两个尺寸参数和一个字符值，创建了一个给定大小的空白屏幕对象。

如我们之前所知的，类内初始值必须使用=的初始化形式（初始化 Screen 的数据成员时所用的）或者花括号括起来的直接初始化形式（初始化 screens 所用的）。

Note

当我们提供一个类内初始值时，必须以符号=或者花括号表示。

275

7.3.1 节练习

练习 7.23: 编写你自己的 Screen 类。

练习 7.24: 给你的 Screen 类添加三个构造函数：一个默认构造函数；另一个构造函数接受宽和高的值，然后将 contents 初始化成给定数量的空白；第三个构造函数接受宽和高的值以及一个字符，该字符作为初始化之后屏幕的内容。

练习 7.25: Screen 能安全地依赖于拷贝和赋值操作的默认版本吗？如果能，为什么？如果不能，为什么？

练习 7.26: 将 Sales_data::avg_price 定义成内联函数。



7.3.2 返回*this的成员函数

接下来我们继续添加一些函数，它们负责设置光标所在位置的字符或者其他任一给定位置的字符：

```
class Screen {
public:
    Screen &set(char);
    Screen &set(pos, pos, char);
    // 其他成员和之前的版本一致
};
inline Screen &Screen::set(char c)
{
    contents[cursor] = c;           // 设置当前光标所在位置的新值
    return *this;                  // 将 this 对象作为左值返回
}
```

```

}
inline Screen &Screen::set(pos r, pos col, char ch)
{
    contents[r*width + col] = ch;    // 设置给定位置的新值
    return *this;                    // 将 this 对象作为左值返回
}

```

和 move 操作一样，我们的 set 成员的返回值是调用 set 的对象的引用（参见 7.1.2 节，第 232 页）。返回引用的函数是左值的（参见 6.3.2 节，第 202 页），意味着这些函数返回的是对象本身而非对象的副本。如果我们把一系列这样的操作连接在一条表达式中的话：

```

// 把光标移动到一个指定的位置，然后设置该位置的字符值
myScreen.move(4,0).set('#');

```

这些操作将在同一个对象上执行。在上面的表达式中，我们首先移动 myScreen 内的光标，然后设置 myScreen 的 contents 成员。也就是说，上述语句等价于

```

myScreen.move(4,0);
myScreen.set('#');

```

如果我们令 move 和 set 返回 Screen 而非 Screen&，则上述语句的行为将大不相同。在此例中等价于：

```

// 如果 move 返回 Screen 而非 Screen&
Screen temp = myScreen.move(4,0);    // 对返回值进行拷贝
temp.set('#');                        // 不会改变 myScreen 的 contents

```

276

假如当初我们定义的回类型不是引用，则 move 的返回值将是 *this 的副本（参见 6.3.2 节，第 201 页），因此调用 set 只能改变临时副本，而不能改变 myScreen 的值。

从 const 成员函数返回 *this

接下来，我们继续添加一个名为 display 的操作，它负责打印 Screen 的内容。我们希望这个函数能和 move 以及 set 出现在同一序列中，因此类似于 move 和 set，display 函数也应该返回执行它的对象的引用。

从逻辑上来说，显示一个 Screen 并不需要改变它的内容，因此我们令 display 为一个 const 成员，此时，this 将是一个指向 const 的指针而 *this 是 const 对象。由此推断，display 的返回类型应该是 const Sales_data&。然而，如果真的令 display 返回一个 const 的引用，则我们将不能把 display 嵌入到一组动作的序列中去：

```

Screen myScreen;
// 如果 display 返回常量引用，则调用 set 将引发错误
myScreen.display(cout).set('*');

```

即使 myScreen 是个非常量对象，对 set 的调用也无法通过编译。问题在于 display 的 const 版本返回的是常量引用，而我们显然无权 set 一个常量对象。



一个 const 成员函数如果以引用的形式返回 *this，那么它的返回类型将是常量引用。

基于 const 的重载

通过区分成员函数是否是 const 的，我们可以对其进行重载，其原因与我们之前根据指针参数是否指向 const（参见 6.4 节，第 208 页）而重载函数的原因差不多。具体说

来, 因为非常量版本的函数对于常量对象是不可用的, 所以我们只能在一个常量对象上调用 `const` 成员函数。另一方面, 虽然可以在非常量对象上调用常量版本或非常量版本, 但显然此时非常量版本是一个更好的匹配。

在下面的这个例子中, 我们将定义一个名为 `do_display` 的私有成员, 由它负责打印 `Screen` 的实际工作。所有的 `display` 操作都将调用这个函数, 然后返回执行操作的对象:

```
class Screen {
public:
    // 根据对象是否是 const 重载了 display 函数
    Screen &display(std::ostream &os)
        { do_display(os); return *this; }
    const Screen &display(std::ostream &os) const
        { do_display(os); return *this; }
private:
    // 该函数负责显示 Screen 的内容
    void do_display(std::ostream &os) const {os << contents;}
    // 其他成员与之前的版本一致
};
```

和我们之前所学的一样, 当一个成员调用另外一个成员时, `this` 指针在其中隐式地传递。因此, 当 `display` 调用 `do_display` 时, 它的 `this` 指针隐式地传递给 `do_display`。而当 `display` 的非常量版本调用 `do_display` 时, 它的 `this` 指针将隐式地从指向非常量的指针转换成指向常量的指针 (参见 4.11.2 节, 第 144 页)。

当 `do_display` 完成后, `display` 函数各自返回解引用 `this` 所得的对象。在非常量版本中, `this` 指向一个非常量对象, 因此 `display` 返回一个普通的 (非常量) 引用; 而 `const` 成员则返回一个常量引用。

当我们在某个对象上调用 `display` 时, 该对象是否是 `const` 决定了应该调用 `display` 的哪个版本:

```
Screen myScreen(5,3);
const Screen blank(5, 3);
myScreen.set('#').display(cout);           // 调用非常量版本
blank.display(cout);                       // 调用常量版本
```

建议: 对于公共代码使用私有功能函数

有些读者可能会奇怪为什么我们要费力定义一个单独的 `do_display` 函数。毕竟, 对 `do_display` 的调用并不比 `display` 函数内部所做的操作简单多少。为什么还要这么做呢? 实际上我们是出于以下原因的:

- 一个基本的愿望是避免在多处使用同样的代码。
- 我们预期随着类的规模发展, `display` 函数有可能变得更加复杂, 此时, 把相应的操作写在一处而非两处的作用就比较明显了。
- 我们很可能在开发过程中给 `do_display` 函数添加某些调试信息, 而这些信息将在代码的最终产品版本中去掉。显然, 只在 `do_display` 一处添加或删除这些信息要更容易一些。

- 这个额外的函数调用不会增加任何开销。因为我们在类内部定义了 `do_display`，所以它隐式地被声明成内联函数。这样的话，调用 `do_display` 就不会带来任何额外的运行时开销。

在实践中，设计良好的 C++ 代码常常包含大量类似于 `do_display` 的小函数，通过调用这些函数，可以完成一组其他函数的“实际”工作。

7.3.2 节练习

练习 7.27: 给你自己的 `Screen` 类添加 `move`、`set` 和 `display` 函数，通过执行下面的代码检验你的类是否正确。

```
Screen myScreen(5, 5, 'X');
myScreen.move(4, 0).set('#').display(cout);
cout << "\n";
myScreen.display(cout);
cout << "\n";
```

练习 7.28: 如果 `move`、`set` 和 `display` 函数的返回类型不是 `Screen&` 而是 `Screen`，则在上一个练习中将会发生什么情况？

练习 7.29: 修改你的 `Screen` 类，令 `move`、`set` 和 `display` 函数返回 `Screen` 并检查程序的运行结果，在上一个练习中你的推测正确吗？

练习 7.30: 通过 `this` 指针使用成员的做法虽然合法，但是有点多余。讨论显式地使用指针访问成员的优缺点。

7.3.3 类类型

每个类定义了唯一的类型。对于两个类来说，即使它们的成员完全一样，这两个类也是两个不同的类型。例如：

```
struct First {
    int mem1;
    int getMem();
};
struct Second {
    int mem1;
    int getMem();
};
First obj1;
Second obj2 = obj1;           // 错误: obj1 和 obj2 的类型不同
```



即使两个类的成员列表完全一致，它们也是不同的类型。对于一个类来说，它的成员和其他任何类（或者任何其他作用域）的成员都不是一回事儿。

我们可以把类名作为类型的名字使用，从而直接指向类类型。或者，我们也可以把类名跟在关键字 `class` 或 `struct` 后面：

```
Sales_data item1;           // 默认初始化 Sales_data 类型的对象
class Sales_data item1;     // 一条等价的声明
```

上面这两种使用类类型的方式是等价的，其中第二种方式从 C 语言继承而来，并且在 C++ 语言中也是合法的。

类的声明

就像可以把函数的声明和定义分离开来一样（参见 6.1.2 节，第 186 页），我们也能仅仅声明类而暂时不定义它：

```
class Screen;                // Screen 类的声明
```

279 这种声明有时被称作**前向声明**（forward declaration），它向程序中引入了名字 Screen 并且指明 Screen 是一种类类型。对于类型 Screen 来说，在它声明之后定义之前是一个**不完全类型**（incomplete type），也就是说，此时我们已知 Screen 是一个类类型，但是不清楚它到底包含哪些成员。

不完全类型只能在非常有限的情景下使用：可以定义指向这种类型的指针或引用，也可以声明（但是不能定义）以不完全类型作为参数或者返回类型的函数。

对于一个类来说，在我们创建它的对象之前该类必须被定义过，而不能仅仅被声明。否则，编译器就无法了解这样的对象需要多少存储空间。类似的，类也必须首先被定义，然后才能用引用或者指针访问其成员。毕竟，如果类尚未定义，编译器也就不清楚该类到底有哪些成员。

在 7.6 节（第 268 页）中我们将描述一种例外的情况：直到类被定义之后数据成员才能被声明成这种类类型。换句话说，我们必须首先完成类的定义，然后编译器才能知道存储该数据成员需要多少空间。因为只有当类全部完成后类才算被定义，所以一个类的成员类型不能是该类自己。然而，一旦一个类的名字出现后，它就被认为是声明过了（但尚未定义），因此类允许包含指向它自身类型的引用或指针：

```
class Link_screen {
    Screen window;
    Link_screen *next;
    Link_screen *prev;
};
```

7.3.3 节练习

练习 7.31：定义一对类 X 和 Y，其中 X 包含一个指向 Y 的指针，而 Y 包含一个类型为 X 的对象。

7.3.4 友元再探

我们的 Sales_data 类把三个普通的非成员函数定义成了友元（参见 7.2.1 节，第 241 页）。类还可以把其他的类定义成友元，也可以把其他类（之前已定义过的）的成员函数定义成友元。此外，友元函数能定义在类的内部，这样的函数是隐式内联的。

类之间的友元关系

举个友元类的例子，我们的 Window_mgr 类（参见 7.3.1 节，第 245 页）的某些成员可能需要访问它管理的 Screen 类的内部数据。例如，假设我们需要为 Window_mgr 添加一个名为 clear 的成员，它负责把一个指定的 Screen 的内容都设为空白。为了完成这一任务，clear 需要访问 Screen 的私有成员；而要想令这种访问合法，Screen 需要

把 Window_mgr 指定成它的友元:

```
class Screen {
    // Window_mgr 的成员可以访问 Screen 类的私有部分
    friend class Window_mgr;
    // Screen 类的剩余部分
};
```

如果一个类指定了友元类, 则友元类的成员函数可以访问此类包括非公有成员在内的所有成员。通过上面的声明, Window_mgr 被指定为 Screen 的友元, 因此我们可以将 Window_mgr 的 clear 成员写成如下的形式:

```
class Window_mgr {
public:
    // 窗口中每个屏幕的编号
    using ScreenIndex = std::vector<Screen>::size_type;
    // 按照编号将指定的 Screen 重置为空白
    void clear(ScreenIndex);
private:
    std::vector<Screen> screens{Screen(24, 80, ' ')};
};
void Window_mgr::clear(ScreenIndex i)
{
    // s 是一个 Screen 的引用, 指向我们想清空的那个屏幕
    Screen &s = screens[i];
    // 将那个选定的 Screen 重置为空白
    s.contents = string(s.height * s.width, ' ');
}
```

一开始, 首先把 s 定义成 screens vector 中第 i 个位置上的 Screen 的引用, 随后利用 Screen 的 height 和 width 成员计算出一个新的 string 对象, 并令其含有若干个空白字符, 最后我们把这个含有很多空白的字符串赋给 contents 成员。

如果 clear 不是 Screen 的友元, 上面的代码将无法通过编译, 因为此时 clear 将不能访问 Screen 的 height、width 和 contents 成员。而当 Screen 将 Window_mgr 指定为其友元之后, Screen 的所有成员对于 Window_mgr 就都变成可见的了。

必须要注意的一点是, 友元关系不存在传递性。也就是说, 如果 Window_mgr 有它自己的友元, 则这些友元并不能理所当然地具有访问 Screen 的特权。



每个类负责控制自己的友元类或友元函数。

令成员函数作为友元

除了令整个 Window_mgr 作为友元之外, Screen 还可以只为 clear 提供访问权限。当把一个成员函数声明成友元时, 我们必须明确指出该成员函数属于哪个类:

```
class Screen {
    // Window_mgr::clear 必须在 Screen 类之前被声明
    friend void Window_mgr::clear(ScreenIndex);
    // Screen 类的剩余部分
};
```

要想令某个成员函数作为友元，我们必须仔细组织程序的结构以满足声明和定义的彼此依赖关系。在这个例子中，我们必须按照如下方式设计程序：

- 首先定义 Window_mgr 类，其中声明 clear 函数，但是不能定义它。在 clear 使用 Screen 的成员之前必须先声明 Screen。
- 接下来定义 Screen，包括对于 clear 的友元声明。
- 最后定义 clear，此时它才可以使用 Screen 的成员。

函数重载和友元

尽管重载函数的名字相同，但它们仍然是不同的函数。因此，如果一个类想把一组重载函数声明成它的友元，它需要对这组函数中的每一个分别声明：

```
// 重载的 storeOn 函数
extern std::ostream& storeOn(std::ostream &, Screen &);
extern BitMap& storeOn(BitMap &, Screen &);
class Screen {
    // storeOn 的 ostream 版本能访问 Screen 对象的私有部分
    friend std::ostream& storeOn(std::ostream &, Screen &);
    // ...
};
```

Screen 类把接受 ostream& 的 storeOn 函数声明成它的友元，但是接受 BitMap& 作为参数的版本仍然不能访问 Screen。



友元声明和作用域

类和非成员函数的声明不是必须在它们的友元声明之前。当一个名字第一次出现在一个友元声明中时，我们隐式地假定该名字在当前作用域中是可见的。然而，友元本身不一定真的声明在当前作用域中（参见 7.2.1 节，第 241 页）。

甚至就算在类的内部定义该函数，我们也必须在类的外部提供相应的声明从而使得函数可见。换句话说，即使我们仅仅是用声明友元的类的成员调用该友元函数，它也必须是被声明过的：

```
struct X {
    friend void f() { /* 友元函数可以定义在类的内部*/ }
    X() { f(); } // 错误：f 还没有被声明
    void g();
    void h();
};
void X::g() { return f(); } // 错误：f 还没有被声明
void f(); // 声明那个定义在 X 中的函数
void X::h() { return f(); } // 正确：现在 f 的声明在作用域中了
```

282

关于这段代码最重要的是理解友元声明的作用是影响访问权限，它本身并非普通意义上的声明。



请注意，有的编译器并不强制执行上述关于友元的限定规则（参见 7.2.1 节，第 241 页）。

7.3.4 节练习

练习 7.32: 定义你自己的 `Screen` 和 `Window_mgr`, 其中 `clear` 是 `Window_mgr` 的成员, 是 `Screen` 的友元。

7.4 类的作用域

每个类都会定义它自己的作用域。在类的作用域之外, 普通的数据和函数成员只能由对象、引用或者指针使用成员访问运算符 (参见 4.6 节, 第 133 页) 来访问。对于类类型成员则使用作用域运算符访问。不论哪种情况, 跟在运算符之后的名字都必须是对应类的成员:

```
Screen::pos ht = 24, wd = 80;           // 使用 Screen 定义的 pos 类型
Screen scr(ht, wd, ' ');
Screen *p = &scr;
char c = scr.get();                     // 访问 scr 对象的 get 成员
c = p->get();                           // 访问 p 所指对象的 get 成员
```

作用域和定义在类外部的成员

一个类就是一个作用域的事实能够很好地解释为什么当我们在类的外部定义成员函数时必须同时提供类名和函数名 (参见 7.1.2 节, 第 230 页)。在类的外部, 成员的名字被隐藏起来了。

一旦遇到了类名, 定义的剩余部分就在类的作用域之内了, 这里的剩余部分包括参数列表和函数体。结果就是, 我们可以直接使用类的其他成员而无须再次授权了。

例如, 我们回顾一下 `Window_mgr` 类的 `clear` 成员 (参见 7.3.4 节, 第 251 页), 该函数的参数用到了 `Window_mgr` 类定义的一种类型:

```
void Window_mgr::clear(ScreenIndex i)
{
    Screen &s = screens[i];
    s.contents = string(s.height * s.width, ' ');
}
```

因为编译器在处理参数列表之前已经明确了我们当前正位于 `Window_mgr` 类的作用域中, 所以不必再专门说明 `ScreenIndex` 是 `Window_mgr` 类定义的。出于同样的原因, 编译器也能知道函数体中用到的 `screens` 也是在 `Window_mgr` 类中定义的。

另一方面, 函数的返回类型通常出现在函数名之前。因此当成员函数定义在类的外部时, 返回类型中使用的名字都位于类的作用域之外。这时, 返回类型必须指明它是哪个类的成员。例如, 我们可能向 `Window_mgr` 类添加一个新的名为 `addScreen` 的函数, 它负责向显示器添加一个新的屏幕。这个成员的返回类型将是 `ScreenIndex`, 用户可以通过它定位到指定的 `Screen`:

```
class Window_mgr {
public:
    // 向窗口添加一个 Screen, 返回它的编号
    ScreenIndex addScreen(const Screen&);
    // 其他成员与之前的版本一致
};
// 首先处理返回类型, 之后我们才进入 Window_mgr 的作用域
```

